

UNIVERSITY OF TECHNOLOGY EINDHOVEN

PROJECT 1

5LSH0 COMPUTER VISION AI AND 3D DATA ANALYSIS

Fundamentals of neural networks

Names

Joost LELIVELD

Student numbers

1609726

June 18, 2026

Contents

1	Multi-Layer Perceptron	2
1.1	Training the MLP	2
1.2	Trainable Parameters	2
1.3	Parameters with convolutional layer	2
1.4	Improved model	3
2	Loss functions	3
2.1	Derivative of output y	3
2.2	Cross-Entropy Loss	4
2.3	Show that $\frac{\partial \mathcal{L}_{dice}}{\partial z} = \frac{(y-1)}{2y} \cdot (1 - \mathcal{L}_{dice})^2$	5
2.4	Focal Loss: $\mathcal{L}_{focal}(y, t) = -(1 - y)^\gamma \cdot t \log(y)$	6
2.5	Show focal loss derivative	6
2.6	Condition for $\frac{\partial \mathcal{L}_{focal}}{\partial y} = \frac{\partial \mathcal{L}_{CE}}{\partial y}$	8
2.7	Applications of loss functions	8
2.8	Behavior of precision and recall for loss functions	8

1 Multi-Layer Perceptron

1.1 Training the MLP

Training Accuracy: 0.90208 Test Accuracy: 0.90130.

The model is optimized on the training set, the parameters of the model are set in a way to get a low value in the loss function. Since the test set contains images different from what the model is optimized for, the accuracy will differ a bit. Since the accuracies are very close to each other, it means that the performance on the test and training set are very similar and the model did not overfit on the images in the training set yet. If I let the model run for more epochs the training and test accuracy would probably differ more, since the model will be even more optimized for training data set.

1.2 Trainable Parameters

Total Trainable Parameters: 109386

This calculation was done in python with the following function:

$$N_{\text{params}} = \sum_{i=1}^M \left(n_{\text{in}}^{(i)} \times n_{\text{out}}^{(i)} + n_{\text{out}}^{(i)} \right).$$

For fully connected layer i with: input dimension of $n_{\text{in}}^{(i)}$, output dimension of $n_{\text{out}}^{(i)}$. $n_{\text{in}}^{(i)} \times n_{\text{out}}^{(i)}$ becomes the size of the weight matrix. The total number of trainable parameters for a layer is given by the size of the weight matrix plus the size of the bias vector. With M fully connected layers this calculation is done M times and summed.

1.3 Parameters with convolutional layer

Using the same approach as the previous question (summation of weights layer by layer), for a single convolution layer followed by a batch normalization (BN) layer. The formula with $k_x \times k_y$ filters (kernel size, width $\times$ height), C_{in} as input channels and C_{out} is number of output channels and the extra C_{out} is for the bias term is given as.

$$N_{\text{params}} = k_x \cdot k_y \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$$

The batch normalization layer has 2 parameters if applied before the activation function (γ and β) for every output channel:

$$N_{\text{params}} = 2 \times C_{\text{out}}$$

Convolution layer + batch normalization:

$$(k_x \cdot k_y \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}) + (2 \times C_{\text{out}}) = k_x \cdot k_y \cdot C_{\text{in}} \cdot C_{\text{out}} + 3 C_{\text{out}}$$

In a convolutional layer the kernel is applied to the all positions of the input, which means the weights are reused. Thus reducing the number of weights by a lot, compared to a MLP where all input nodes are connected to all output nodes. Applying kernels also is good for preserving dimensional structures of the data. The convolutional layer is the same as a fully connected layer when the input is flattened and then the kernel has the same size as the input. Thus connecting all input units to all output units. A 1×1 kernel could have the same effect by flattening the channel dimension.

1.4 Improved model

Test accuracy: 0.9882. I have implemented 4 main changes to improve the previous model:

- **Data augmentation:** I randomly transformed the training images to make the dataset more diverse. By translating and rotating parts of the image the model becomes more generalized. So the model is less specific to the specific patterns in the training data set. This could also help with enlarging the dataset to have more training samples, however that has its downsides.
- **Batch Normalization:** This stabilizes and accelerates the training process by normalizing the activations of a layer to have zero mean and unit variance. It prevents internal covariate shift of the weights and reduces the chance of vanishing gradients. This way higher learning rates can be used, which leads to faster convergence. It can also function as a regularization factor to prevent overfitting.
- **Dropout:** Introducing noise in to the learning process by randomly disabling neurons during training helps with preventing overfitting on the training data. And that could lead to a higher accuracy on the test set.
- **Adam optimizer:** Instead of using standard gradient descent I used the Adam optimizer. The learning rate in this optimizer is adaptive for each parameter based on the estimates of the mean and variance of gradients. This greatly speeds up the training process.

With unlimited resources for computation, performance improvements could be:

- Deeper architectures could capture more complex patterns in the images, for example models like ResNet, DenseNet, or Vision Transformers.
- Ensemble models combine prediction using boosting, by using predictions from multiple models overall accuracy could be increased.
- Hyperparameter optimization will help find the best accuracy for the current solution as well as for other models. Right now the learning rate was adjusted based on trying with a few different ones, but it was not optimized. But other parameters that could be changed are the batch size, number of layers, dropout rate, kernel size and optimizer.
- Attention mechanisms help to concentrate on the most important parts of the image. With self attention the spatial dependencies can be found, which can help with classifying the images.

2 Loss functions

2.1 Derivative of output y

Y is given as the softmax equation:

$$y = \sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

We want to calculate $\frac{\partial y_i}{\partial z_j}$. There are two distinct cases for the derivative.

Proof for Case 1: $i = j$

Using the quotient rule:

$$\frac{\partial y_i}{\partial z_i} = \frac{\frac{\partial}{\partial z_i} e^{z_i} \cdot \left(\sum_{j=1}^K e^{z_j} \right) - e^{z_i} \cdot \frac{\partial}{\partial z_i} \left(\sum_{j=1}^K e^{z_j} \right)}{\left(\sum_{j=1}^K e^{z_j} \right)^2}.$$

Simplify the derivatives:

$$\frac{\partial}{\partial z_i} \left(\sum_{j=1}^K e^{z_j} \right) = e^{z_i} \quad (\text{only the } i\text{-th term contributes}).$$

Substitute this back:

$$\frac{\partial y_i}{\partial z_i} = \frac{e^{z_i} \left(\sum_{j=1}^K e^{z_j} \right) - e^{z_i} \cdot e^{z_i}}{\left(\sum_{j=1}^K e^{z_j} \right)^2}.$$

Factor out e^{z_i} :

$$\frac{\partial y_i}{\partial z_i} = \frac{e^{z_i} \left(\sum_{j=1}^K e^{z_j} - e^{z_i} \right)}{\left(\sum_{j=1}^K e^{z_j} \right)^2}.$$

Substitute $y_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$:

$$\frac{\partial y_i}{\partial z_i} = y_i (1 - y_i).$$

Proof for Case 2: $i \neq j$

Using the quotient rule again:

$$\frac{\partial y_i}{\partial z_j} = \frac{0 \cdot \left(\sum_{j=1}^K e^{z_j} \right) - e^{z_i} \cdot \frac{\partial}{\partial z_j} \left(\sum_{j=1}^K e^{z_j} \right)}{\left(\sum_{j=1}^K e^{z_j} \right)^2}.$$

Since $\frac{\partial}{\partial z_j} \left(\sum_{j=1}^K e^{z_j} \right) = e^{z_j}$, we have:

$$\frac{\partial y_i}{\partial z_j} = \frac{-e^{z_i} \cdot e^{z_j}}{\left(\sum_{j=1}^K e^{z_j} \right)^2}.$$

Substitute $y_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$ and $y_j = \frac{e^{z_j}}{\sum_{j=1}^K e^{z_j}}$:

$$\frac{\partial y_i}{\partial z_j} = -y_i y_j$$

This gives the final solution:

$$\text{When } i = k : \frac{\partial y_i}{\partial z_i} = y_i (1 - y_i),$$

$$\text{When } i \neq k : \frac{\partial y_i}{\partial z_j} = -y_i y_j.$$

2.2 Cross-Entropy Loss

$$\mathcal{L}_{ce}(y, t) = -(t \cdot \log(y) + (1 - t) \cdot \log(1 - y)),$$

where t is the truth label, and y the result of the softmax function.

Using the hint:

$$\frac{\partial \mathcal{L}_{ce}}{\partial z} = \frac{\partial \mathcal{L}_{ce}}{\partial y} \cdot \frac{\partial y}{\partial z}.$$

Using the solution from previous question:

$$\frac{\partial y_i}{\partial z_j} = \begin{cases} y_i(1 - y_i), & \text{if } i = j, \\ -y_i y_j, & \text{if } i \neq j. \end{cases}$$

$$\frac{\partial \mathcal{L}_{ce}}{\partial y} = - \left(\frac{t}{y} - \frac{1-t}{1-y} \right).$$

In the condition when $i = j$, by substituting the expressions we get:

$$\frac{\partial \mathcal{L}_{ce}}{\partial z_j} = - \left(\frac{t}{y_j} - \frac{1-t}{1-y_j} \right) y_j(1 - y_j)$$

Solve by getting the denominator to $y_j(1 - y_j)$:

$$\frac{t}{y_j} - \frac{1-t}{1-y_j} = \frac{t(1-y_j) - (1-t)y_j}{y_j(1-y_j)}$$

Numerator becomes:

$$t(1-y_j) - (1-t)y_j = t - ty_j - y_j + ty_j = t - y_j$$

Thus, by substituting it back

$$\frac{\partial \mathcal{L}_{ce}}{\partial z_j} = - \left(\frac{t - y_j}{y_j(1 - y_j)} \right) y_j(1 - y_j)$$

Now $y_j(1 - y_j)$ can be canceled out and final answer becomes:

$$\frac{\partial \mathcal{L}_{ce}}{\partial z_j} = y_j - t$$

2.3 Show that $\frac{\partial \mathcal{L}_{dice}}{\partial z} = \frac{(y-1)}{2y} \cdot (1 - \mathcal{L}_{dice})^2$

We can use the same methods as the previous question. First start with the loss function:

$$\mathcal{L}_{dice}(y, t) = 1 - \frac{2yt}{y+t}.$$

Then again using the hint, we know that we need to find $\frac{\partial \mathcal{L}_{dice}}{\partial y}$:

$$\frac{\partial \mathcal{L}_{dice}}{\partial z} = \frac{\partial \mathcal{L}_{dice}}{\partial y} \cdot \frac{\partial y}{\partial z}.$$

Using the quotient rule:

$$\frac{\partial \mathcal{L}_{dice}}{\partial y} = - \frac{2t(y+t) - 2yt}{(y+t)^2} = - \frac{2t \cdot t}{(y+t)^2} = - \frac{2t^2}{(y+t)^2}.$$

Substituting $\frac{\partial y}{\partial z} = y(1 - y)$:

$$\frac{\partial \mathcal{L}_{dice}}{\partial z} = \left(- \frac{2t^2}{(y+t)^2} \right) \cdot y(1 - y).$$

$$1 - \mathcal{L}_{dice} = \frac{2yt}{y+t}.$$

We need to have $(y+t)^2$ as denominator for substituting:

$$(1 - \mathcal{L}_{dice})^2 = \left(\frac{2yt}{y+t} \right)^2 = \frac{4y^2t^2}{(y+t)^2}$$

$$\left(-\frac{2t^2}{(y+t)^2} \right) = -\frac{(1 - \mathcal{L}_{dice})^2}{2y^2}$$

Substitute this result into the derivative:

$$\frac{\partial \mathcal{L}_{dice}}{\partial z} = -\frac{1}{2y^2} \cdot (1 - \mathcal{L}_{dice})^2 \cdot y(1-y).$$

$$\frac{\partial \mathcal{L}_{dice}}{\partial z} = \frac{(y-1)}{2y} \cdot (1 - \mathcal{L}_{dice})^2.$$

2.4 Focal Loss: $\mathcal{L}_{focal}(y, t) = -(1-y)^\gamma \cdot t \log(y)$

The binary classification focal loss applies when the true label $t \in \{0, 1\}$. So the two cases to give the function for are $t = 1$ and $t = 0$.

$t = 1$:

$$\mathcal{L}_{focal} = -(1-y)^\gamma \cdot \log(y).$$

$t = 0$:

$$\mathcal{L}_{focal} = -y^\gamma \cdot (1-t) \log(1-y).$$

Substitute $t = 0$:

$$\mathcal{L}_{focal} = -y^\gamma \log(1-y).$$

Combining the cases:

$$\mathcal{L}_{focal}(y, t) = -t \cdot (1-y)^\gamma \log(y) - (1-t) \cdot y^\gamma \log(1-y).$$

2.5 Show focal loss derivative

Show that:

$$\frac{\partial \mathcal{L}_{focal}}{\partial y} = -\frac{1}{y \cdot (1-y)} \left[t \cdot (1-y)^\gamma \cdot (\gamma \cdot \mathcal{E}_y + 1 - y) + (1-t) \cdot y^\gamma \cdot (\gamma \cdot \mathcal{E}_{1-y} + y) \right],$$

when $\mathcal{E}_p = -p \log(p)$

Using the binary version from the previous question we take derivatives from both terms. We can take t as a constant and use the product rule.

First term:

$$\begin{aligned} \frac{\partial}{\partial y} [-t \cdot (1-y)^\gamma \log(y)] &= -t \left[\gamma \cdot (1-y)^{\gamma-1} \cdot (-1) \cdot \log(y) + (1-y)^\gamma \cdot \frac{1}{y} \right] \\ &= t \cdot \gamma \cdot (1-y)^{\gamma-1} \log(y) - t \cdot (1-y)^\gamma \cdot \frac{1}{y}. \end{aligned}$$

Second term:

$$\begin{aligned}\frac{\partial}{\partial y} [-(1-t) \cdot y^\gamma \log(1-y)] &= -(1-t) \left[\gamma \cdot y^{\gamma-1} \cdot \log(1-y) + y^\gamma \cdot \frac{-1}{1-y} \right] \\ &= -(1-t) \cdot \gamma \cdot y^{\gamma-1} \log(1-y) + (1-t) \cdot y^\gamma \cdot \frac{1}{1-y}.\end{aligned}$$

Now we need to simplify and express in terms of $\mathcal{E}_p = -p$

$$\mathcal{E}_y = -y \log(y) \quad \text{and} \quad \mathcal{E}_{1-y} = -(1-y) \log(1-y).$$

This gives:

$$\log(y) = -\frac{\mathcal{E}_y}{y}, \quad \log(1-y) = -\frac{\mathcal{E}_{1-y}}{1-y}.$$

Substituting $\log(y)$ and $\log(1-y)$ into the derivative:

$$\frac{\partial \mathcal{L}_{focal}}{\partial y} = t \cdot \gamma \cdot (1-y)^{\gamma-1} \cdot \left(-\frac{\mathcal{E}_y}{y} \right) - t \cdot (1-y)^\gamma \cdot \frac{1}{y} - (1-t) \cdot \gamma \cdot y^{\gamma-1} \cdot \left(-\frac{\mathcal{E}_{1-y}}{1-y} \right) + (1-t) \cdot y^\gamma \cdot \frac{1}{1-y}.$$

For the first term:

$$\begin{aligned}t \cdot \gamma \cdot (1-y)^{\gamma-1} \log(y) - t \cdot (1-y)^\gamma \cdot \frac{1}{y} &= t \cdot \gamma \cdot (1-y)^{\gamma-1} \cdot -\frac{\mathcal{E}_y}{y} - t \cdot (1-y)^\gamma \cdot \frac{1}{y} \\ &= \frac{-1}{y} [t \cdot \gamma \cdot (1-y)^{\gamma-1} \mathcal{E}_y + t \cdot (1-y)^\gamma] \\ &= \frac{-1}{y \cdot (1-y)} [t \cdot \gamma \cdot (1-y)^\gamma \mathcal{E}_y + t \cdot (1-y)^\gamma \cdot (1-y)] \\ &= \frac{-1}{y \cdot (1-y)} [t \cdot (1-y)^\gamma \cdot (\gamma \cdot \mathcal{E}_y + (1-y))].\end{aligned}$$

For the second term:

$$\begin{aligned}-(1-t) \cdot \gamma \cdot y^{\gamma-1} \log(1-y) + (1-t) \cdot y^\gamma \cdot \frac{1}{1-y} &= -(1-t) \cdot \gamma \cdot y^{\gamma-1} \cdot -\frac{\mathcal{E}_{1-y}}{1-y} + (1-t) \cdot y^\gamma \cdot \frac{1}{1-y} \\ &= \frac{-1}{1-y} [(1-t) \cdot \gamma \cdot y^{\gamma-1} \cdot -\mathcal{E}_{1-y} - (1-t) \cdot y^\gamma] \\ &= \frac{-1}{y \cdot (1-y)} [(1-t) \cdot \gamma \cdot y^\gamma \cdot -\mathcal{E}_{1-y} - (1-t) \cdot y^\gamma \cdot y] \\ &= \frac{-1}{y \cdot (1-y)} [(1-t) \cdot \gamma \cdot y^\gamma \cdot \mathcal{E}_{1-y} + (1-t) \cdot y^\gamma \cdot y] \\ &= \frac{-1}{y \cdot (1-y)} [(1-t) \cdot y^\gamma \cdot (\gamma \cdot \mathcal{E}_{1-y} + y)].\end{aligned}$$

The only thing left is to combine the first and second term back into the other formula:

$$\frac{\partial \mathcal{L}_{focal}}{\partial y} = -\frac{1}{y \cdot (1-y)} [t \cdot (1-y)^\gamma \cdot (\gamma \cdot \mathcal{E}_y + 1-y) + (1-t) \cdot y^\gamma \cdot (\gamma \cdot \mathcal{E}_{1-y} + y)].$$

2.6 Condition for $\frac{\partial \mathcal{L}_{focal}}{\partial y} = \frac{\partial \mathcal{L}_{CE}}{\partial y}$

$$\frac{\partial \mathcal{L}_{focal}}{\partial y} = \frac{\partial \mathcal{L}_{CE}}{\partial y}$$

$$\frac{-1}{y \cdot (1-y)} [t \cdot (1-y)^\gamma \cdot (\gamma \cdot \mathcal{E}_y + 1-y) + (1-t) \cdot y^\gamma \cdot (\gamma \cdot \mathcal{E}_{1-y} + y)] = -\frac{t}{y} + \frac{1-t}{1-y}.$$

Is hard to solve, a more intuitive solution holds when $\gamma = 0$. Then the modulating factors $(1-y)^\gamma$ and y^γ become 1, and the focal loss becomes equal to the standard cross-entropy loss:

$$-t \cdot (1-y)^\gamma \log(y) - (1-t) \cdot y^\gamma \log(1-y) = -(t \cdot \log(y) + (1-t) \cdot \log(1-y))$$

when $\gamma = 0$

$$\mathcal{L}_{focal} = \mathcal{L}_{CE}.$$

Which of course means their gradients will also be equal. In this case when γ equals 0 the modulating factor is small. Meaning that all samples are regarded just as important, independent of the confidence in the predictions. So the training focuses on both hard and easy samples. For a larger γ it is of course the other way around, for confident samples y is close to t . Causing the modulating term to be very small. Hard samples where y is further from t will have large modulating terms and will have more influence during training. This could prevent the training to steer away from focussing on easy samples.

2.7 Applications of loss functions

- Cross Entropy Loss: This is the most common and straight to the point loss function. It is more simple to implement and better interpretable, calculating the difference between the predicted and true labels, penalizing based on confidence. It works well on datasets that are balanced and less additional factors need to be taken into account. So it is used in all types of applications: NLP, image classification etc.
- Dice Loss: It is most commonly used in cases for image segmentation with imbalanced datasets. By focussing on the Dice-Sørensen coefficient, which is a measure for overlap between two regions (in case of loss function ground truth regions and the predicted regions), it is good at delineating regions. For example when positive regions that need to be detected are very small compared to the negative region around it. Which makes it good for medical imaging.
- Focal Loss: It is good for imbalanced datasets by applying the modulating term in order to focus learning on hard misclassified examples. The modulating term applied to cross entropy loss can lower the contribution of easy examples and shift the focus to harder samples. In object detection it is common to have these imbalanced datasets, so then focal loss is used to have better performance on less represented classes.

2.8 Behavior of precision and recall for loss functions

- Cross-entropy loss:
 $\mathcal{L}_{ce}(y, t) = -(t \cdot \log(y) + (1-t) \cdot \log(1-y))$, with $\frac{\partial \mathcal{L}_{ce}}{\partial z_j} = y_j - t$ for $i = j$

It tries to balance F_P and F_N by minimizing the log probability of incorrect predictions. The derivative is very straightforward for the case that is correct. If it was correctly predicted with full confidence the gradient is 0 and thus it will not shift, for other cases it will shift in the direction where it will become more confident. This optimizes precision and recall when the dataset is balanced. It does not target FP or FN specifically and thus relies on a balanced dataset.

- Dice Loss:

$\mathcal{L}_{dice}(y, t) = 1 - \frac{2yt}{y+t}$, with $\frac{\partial \mathcal{L}_{dice}}{\partial z} = \frac{(y-1)}{2y} \cdot (1 - \mathcal{L}_{dice})^2$ As mentioned before the dice loss measures the overlap between the predicted regions and the truth regions. The derivative shows that the loss decreases when y and t overlap, which is the case for true positives. This way False negatives are also minimized, which means the function focuses on improving the recall. However for the precision there is a FP term in the denominator. False positives are less penalized in the function. The $(y-1)$ and $y+t$ term increase the loss when y becomes too large (too many positive predictions). These are however both more indirect penalties than the loss-decrease for True positives. So the precision will be less with this loss function, since there will be more false positives.

- Focal Loss:

$\mathcal{L}_{focal}(y, t) = -t \cdot (1-y)^\gamma \log(y) - (1-t) \cdot y^\gamma \log(1-y)$, with

$$\frac{\partial \mathcal{L}_{focal}}{\partial y} = -\frac{1}{y \cdot (1-y)} [t \cdot (1-y)^\gamma \cdot (\gamma \cdot \mathcal{E}_y + 1 - y) + (1-t) \cdot y^\gamma \cdot (\gamma \cdot \mathcal{E}_{1-y} + y)] .$$

As mentioned before the focal loss assigns a higher penalty to inputs that were predicted with less confidence, visible in the terms $(1-y)^\gamma$ and y^γ . This penalizes F_P and F_N equally, thus balancing precision and recall while focusing on hard examples. If there is a dataset with a lot of positives and less negatives, and the negatives are harder to classify, more focus will be on classifying the more difficult negatives thus leading to less false negatives. However this is dependent on the dataset, so in general the focal loss focuses on both precision and recall.